Activity 1 Hashing and Encryption Decision Cases

Student: Laurence Matthew B Manuel **Section:** BSIT 3A **Project:** Project 29 Thesis Consultation Reminder System
Assessment: Week 14 Architecture and Decision Activity **Completed:** 8 November 2026 Asia/Manila

Purpose and decision authority

This activity reviews five self-created examples based on the approved course scenario because the supplied Module 14 gives the task but does not enumerate five instructor cases. The prototype coordinator may accept the controlled technical result; the instructor retains assessment authority. The analysis assumes fictional records, a Laravel server trusted to process plaintext, an untrusted public Flutter client, a protected server keyring, and a maintained HTTPS endpoint.

Five proposed uses and corrections

Example proposal	Security objective	Assessment and corrected design	Operational assumptions and safe adverse behavior
1. Store user passwords with SHA-256 so login can compare hashes.	Verify a password without retaining its plaintext.	Inappropriate. A fast general-purpose hash makes guessing inexpensive. Use Laravel `Hash` with an adaptive password hash and per-password salt. Do not encrypt passwords for recovery.	Hash parameters and framework must be maintained. A wrong password returns the same generic 401 path without revealing whether the account exists.
2. Encrypt the consultation payload with AES-CBC and send the IV beside it.	Keep the agenda and schedule confidential.	Incomplete because encryption alone may not detect modification. Use AES-256-GCM with a random 96-bit nonce, 128-bit tag, and authenticated metadata. Sending the nonce is correct; reusing it with the same key is not.	The server key is random, protected, backed up, and rotated. Altered ciphertext or metadata is rejected before plaintext or receipt creation.
3. Use the AES-GCM envelope instead of HTTPS because the sensitive fields are already encrypted.	Protect browser-to-server traffic and endpoint identity.	Inappropriate. The envelope does not protect credentials, session tokens, URLs, response metadata, or server identity. Use certificate-validated HTTPS and keep the envelope for the controlled payload exercise.	Clients trust only the intended CA and hostname. A missing trust anchor produces a certificate verification failure; no warning bypass is accepted.
4. Add a plain SHA-256 hash of every audit record to prove logs were not changed.	Detect unauthorized log modification.	Weak if an attacker can change both record and hash. In a multi-system design, use a keyed MAC with a separately protected log-integrity key or an append-only signed service. The Week 14 prototype instead limits and protects audit data and does not claim cryptographic log immutability.	Verification keys, retention, canonicalization, and alert ownership must be defined. A broken chain or invalid MAC would quarantine the batch and trigger review rather than silently accepting it.
5. Use the shared AES key as a digital signature so the adviser can prove the coordinator sent the envelope.	Authenticity with possible nonrepudiation.	Inappropriate. Either shared-key holder can create a valid tag. If independent public verification is required, use an established digital-signature scheme such as Ed25519 with distinct private keys and governed public-key distribution. This is not required for the single-server prototype.	Private signing keys need separate ownership, rotation, revocation, and identity binding. An invalid signature must reject the message, but signature validity alone must not grant application authorization.

Approach comparison

Criterion	Selected symmetric AEAD plus HTTPS	Hybrid encryption and signatures
Functional fit	Directly meets the bounded one-server sender/receiver exercise.	Supports independent systems and public verification.
Security	Confidentiality and integrity with TLS, expiry, recipient binding, and replay receipt.	Strong identity separation if certificate and key governance are correct.

Criterion	Selected symmetric AEAD plus HTTPS	Hybrid encryption and signatures
Reliability and performance	Installed OpenSSL path, small envelopes, few failure dependencies.	More keys, formats, certificate failures, and operational dependencies.
Implementation and skills	Individually explainable in the verified PHP runtime.	Higher implementation and defense burden for no current requirement.
Disadvantage	Shared key cannot provide nonrepudiation and requires disciplined nonce/rotation practice.	Expanded key lifecycle and more ways to misconfigure trust.

The symmetric AEAD design was selected. It is not production ready and must be reconsidered if independent organizations, public verification, or nonrepudiation becomes a real requirement.

Repeatable verification

Run `C:\xampp\php\php.exe artisan test --filter=SecureExchangeTest` from `backend`, then run `submissions/week-14/tests/Invoke-Week14Security.ps1` while the controlled HTTPS services are active. The actual 8 November run accepted one authentic envelope and rejected replay, altered ciphertext, an unauthorized role, and invalid input with the expected HTTP status. The authentic expired-envelope, wrong-key, and missing-key recovery cases are covered by Laravel clock/configuration-controlled tests.

One-page interpretation

Observed facts. PHP 8.2.12 exposes OpenSSL and not Sodium. Laravel's framework password hashing already protects fixture passwords. Twenty-one backend tests passed with 199 assertions after the secure-exchange extension. Five live HTTPS cases matched expected status codes, TLS 1.3 negotiated `TLS_AES_256_GCM_SHA384`, and a client without the controlled CA rejected the certificate. Browser evidence showed coordinator creation, adviser receipt, and replay rejection. No key or token appears in the saved results.

Reasoned conclusions. A control must match its objective. Password hashing, authenticated payload encryption, HTTPS, message replay control, and digital signatures solve different problems and are not interchangeable. AES-256-GCM is justified here because it supplies confidentiality and modification detection through an installed library, while TLS authenticates the endpoint and protects credentials and sessions. The unique receipt and expiry rules are necessary because valid encryption alone does not establish freshness. The shared key is acceptable only within the single-server exercise and cannot support a nonrepudiation claim.

Recommendations. Keep the selected design for the bounded Week 14 prototype. Before broader use, assign operational owners, shorten or idle-expire sessions, move secrets to an approved protected store, use a maintained certificate service, add time-drift monitoring, and decide whether independent system identities justify asymmetric keys and signatures. Perform a manual screen-reader and keyboard audit because automated Flutter widget checks do not prove full assistive-technology usability.

Acknowledgment

The implementation uses Laravel, Flutter, PHP OpenSSL, Python cryptography for local test certificates, and AI-assisted drafting and review. Exact institutional acknowledgment wording remains subject to instructor guidance.